Explanation of Important SciPy Functions Used in ML

1. Statistics (`scipy.stats`)

A. `zscore()`

Purpose: Standardizes data by converting values into Z-scores.

Syntax:

```
zscore(data, axis=0, ddof=0)
```

Parameters:

- `data`: Input dataset or array.
- `axis=0`: Calculates Z-scores column-wise.
- `ddof=0`: Delta degrees of freedom used to calculate standard deviation. Default is 0.

Example:

```
zscore([10, 20, 30, 40, 50])
```

B. `describe()`

Purpose: Provides a statistical summary of a dataset.

Syntax:

```
describe(data, axis=0)
```

Parameters:

- `data`: Input dataset.
- `axis=0`: Calculates statistics column-wise.
- `ddof=1`: Degrees of freedom for variance calculation (default).
- `bias=True`: Applies biased statistical estimators for skewness and kurtosis.

Returns:

- `nobs`: Number of observations.
- `minmax`: Minimum and maximum values.
- `mean`: Average value.
- `variance`: Variance of data.
- `skewness`: Measures data asymmetry.
- `kurtosis`: Measures the distribution's tail shape.

C. `norm()`

Purpose: Represents a Normal (Gaussian) probability distribution.

Syntax:

```
norm.pdf(x, loc=0, scale=1)
```

Parameters:

- `x`: Value at which probability density is calculated.
- `loc`: Mean of the distribution (default = 0).
- `scale`: Standard deviation of the distribution (default = 1).

D. `pdf()`

Purpose: Probability Density Function. Calculates the probability density at a particular value.

Syntax:

```
norm.pdf(x, loc=0, scale=1)
```

Parameters:

- `x`: Input value.
- `loc`: Mean of the normal distribution.
- `scale`: Standard deviation of the normal distribution.

Example:

```
norm.pdf(0, loc=0, scale=1)
```

Output: Approximately `0.3989`.

E. `np.linspace()`

Purpose: Generates evenly spaced numbers within a specified interval.

Syntax:

```
np.linspace(start, stop, num)
```

Parameters:

- `start`: Starting value.
- `stop`: Ending value.
- `num`: Number of values to generate (default = 50).

F. `sns.histplot()`

Purpose: Creates a histogram to visualize the distribution of data.

Syntax:

```
sns.histplot(data, kde=True, stat="density")
```

Parameters:

- `data` : Dataset to visualize.
- `kde=True` : Adds a smooth density curve.
- `stat="density"` : Displays density instead of frequency counts.

G. `plt.plot()`

Purpose: Creates a line plot.

Syntax:

```
plt.plot(x, y, label="Normal Distribution")
```

Parameters:

- `x` : X-axis values.
- `y` : Y-axis values.
- `label` : Name of the line displayed in the legend.

H. `plt.legend()` and `plt.show()`

- `plt.legend()` : Displays labels of plotted lines.
- `plt.show()` : Displays the plot.

2. Correlation (`scipy.stats`)

A. `pearsonr()`

Purpose: Calculates the Pearson correlation coefficient to measure the linear relationship between two variables.

Syntax:

```
pearsonr(x, y)
```

Parameters:

- `x` : First dataset.
- `y` : Second dataset.

Returns:

- `statistic` : Pearson correlation coefficient (r), ranging from -1 to +1.
- `pvalue` : Indicates statistical evidence against zero correlation.

B. `spearmanr()`

Purpose: Calculates Spearman's rank correlation to measure the strength of a monotonic relationship between two variables.

Syntax:

```
spearmanr(x, y)
```

Parameters:

- `x` : First dataset.
- `y` : Second dataset.

Returns:

- `statistic` : Spearman correlation coefficient.
- `pvalue` : Statistical significance of the correlation.

C. `iqr()`

Purpose: Calculates the Interquartile Range (IQR), which measures the spread of the middle 50% of data.

Syntax:

```
iqr(data, axis=None, rng=(25, 75))
```

Parameters:

- `data` : Input dataset.
- `axis=None` : Calculates IQR across the entire dataset.
- `rng=(25, 75)` : Uses the 25th and 75th percentiles.

D. `sns.regplot()`

Purpose: Creates a scatter plot with a regression line to visualize the relationship between two variables.

Syntax:

```
sns.regplot(x=x, y=y)
```

Parameters:

- `x` : Independent variable.
- `y` : Dependent variable.
- `data` : Dataset (optional).
- `fit_reg=True` : Displays the regression line (default).
- `ci=95` : Confidence interval percentage (default).

E. `plt.show()`

- Displays the scatter plot and regression line.

3. Optimization (`scipy.optimize`)

A. `minimize()`

Purpose: Finds the minimum value of a mathematical function.

Syntax:

```
minimize(fun, x0, bounds=None)
```

Parameters:

- `fun`: Objective function to minimize.
- `x0`: Initial guess for the variable.
- `bounds`: Defines lower and upper limits for variables.

Important result attributes:

- `result.x`: Value of the variable at the minimum.
- `result.fun`: Minimum function value.
- `result.success`: Indicates whether optimization was successful.

Understanding the lambda function:

```
lambda x: (x[0] - 5)**2
```

- `lambda`: Creates a small anonymous function without using `def`.
- `x`: Input variable.
- `x[0]`: First element of the input array.
- `(x[0] - 5)**2`: Mathematical expression to minimize.

B. `least_squares()`

Purpose: Finds the best-fit solution by minimizing the sum of squared residuals.

Syntax:

```
least_squares(fun, x0)
```

Parameters:

- `fun`: Function that calculates residuals (errors).
- `x0`: Initial guess for unknown variables.

Important result attributes:

- `result.x`: Optimized variable values.
- `result.cost`: Half the sum of squared residuals.
- `result.fun`: Final residual values.
- `result.success`: Indicates successful optimization.

4. Linear Algebra (`scipy.linalg`)

A. `svd()`

Purpose: Performs Singular Value Decomposition (SVD) of a matrix. It is useful in dimensionality reduction and data compression.

Syntax:

```
U, S, Vt = svd(A)
```

Parameters:

- `A` : Input matrix.
- `U` : Left singular vectors.
- `S` : Singular values.
- `Vt` : Transpose of right singular vectors.

Important:

- Singular values represent the importance of different components of the matrix.
- SVD is commonly used in PCA and dimensionality reduction.

B. `eigh()`

Purpose: Calculates eigenvalues and eigenvectors of a real symmetric or complex Hermitian matrix.

Syntax:

```
values, vectors = eigh(A)
```

Parameters:

- `A` : Input symmetric or Hermitian matrix.

Returns:

- `values` : Eigenvalues arranged in ascending order.
- `vectors` : Corresponding eigenvectors stored as columns.

C. `solve()`

Purpose: Solves a system of linear equations.

Syntax:

```
solve(A, b)
```

Parameters:

- `A` : Coefficient matrix.

- `b` : Right-hand side values.

Example mathematical representation:

\[Ax=b \]

Returns:

- Solution values of unknown variables.

D. `np.array()`

Purpose: Creates a NumPy array or matrix.

Syntax:

```
np.array([[4, 1], [1, 3]])
```

Parameters:

- `[[4, 1], [1, 3]]` : Input data used to create a 2D array.

5. Interpolation (`scipy.interpolate`)

A. `interp1d()`

Purpose: Performs one-dimensional interpolation to estimate unknown values between known data points.

Syntax:

```
f = interp1d(x, y, kind="linear")
```

Parameters:

- `x` : Known X-axis values.
- `y` : Corresponding Y-axis values.
- `kind="linear"` : Uses linear interpolation.
- `kind="nearest"` : Uses the nearest data point.
- `kind="cubic"` : Uses cubic interpolation.

Important:

- `f` : Stores the interpolation function.
- `f(x_new)` : Calculates interpolated values for new X-axis points.

B. `griddata()`

Purpose: Interpolates scattered data points to estimate values on a regular grid.

Syntax:

```
griddata(points, values, xi, method="linear")
```

Parameters:

- `points` : Coordinates of known data points.
- `values` : Values corresponding to known points.
- `xi` : Coordinates where new values are required.
- `method="linear"` : Uses linear interpolation.
- `method="nearest"` : Uses the nearest data point.
- `method="cubic"` : Uses cubic interpolation (for 2D data).

C. `np.mgrid`

Purpose: Creates coordinate grids for multidimensional data.

Syntax:

```
xi, yi = np.mgrid[0:1:30j, 0:1:30j]
```

Parameters:

- `0:1` : Defines the coordinate range.
- `30j` : Generates 30 evenly spaced points along that dimension.

D. `plt.scatter()`

Purpose: Creates a scatter plot to display individual data points.

Syntax:

```
plt.scatter(x, y, color="red")
```

Parameters:

- `x` : X-axis coordinates.
- `y` : Y-axis coordinates.
- `color` : Specifies point color.

E. `plt.contourf()`

Purpose: Creates a filled contour plot to visualize values across a 2D grid.

Syntax:

```
plt.contourf(xi, yi, zi, levels=15)
```

Parameters:

- `xi` : X-axis grid coordinates.

- `yi` : Y-axis grid coordinates.
- `zi` : Interpolated values.
- `levels=15` : Number of contour levels.

F. `plt.colorbar()`

- Adds a color scale to the plot.
- Helps identify which colors represent which numerical values.

G. `plt.show()`

- Displays the interpolation plot.

6. Fourier Transform and Signal Processing

A. `fft()`

Purpose: Performs Fast Fourier Transform to convert a signal from the time domain into the frequency domain.

Syntax:

```
fft(signal)
```

Parameters:

- `signal` : Input time-domain signal.

Returns:

- Complex frequency-domain values representing the signal's frequency components.

B. `np.linspace()`

Purpose: Generates evenly spaced time values.

Syntax:

```
np.linspace(start, stop, num, endpoint=False)
```

Parameters:

- `start` : Starting time.
- `stop` : Ending time.
- `num` : Number of samples.
- `endpoint=False` : Excludes the final value from the generated array.

C. `np.sin()`

Purpose: Calculates the sine of each element in an array.

Syntax:

```
np.sin(x)
```

Parameters:

- `x`: Input angles in radians.

D. `np.pi`

- Represents the mathematical constant π (approximately 3.14159).

E. `np.fft.fftfreq()`

Purpose: Generates the frequency values corresponding to the FFT output.

Syntax:

```
np.fft.fftfreq(n, d)
```

Parameters:

- `n`: Number of samples.
- `d`: Sample spacing (time difference between consecutive samples).

Important: This function belongs to NumPy, not the SciPy import used for `fft()`.

F. `np.abs()`

Purpose: Calculates the absolute value or magnitude of numbers.

Syntax:

```
np.abs(result)
```

Use in FFT:

- Calculates the magnitude of complex FFT values.
- Helps visualize the strength of different frequency components.

G. `find_peaks()`

Purpose: Detects local peaks in a signal or dataset.

Syntax:

```
find_peaks(y, prominence=0.2)
```

Parameters:

- `y`: Input signal.
- `prominence`: Minimum prominence required for a peak.

- `height` : Minimum peak height.
- `distance` : Minimum horizontal distance between neighboring peaks.
- `width` : Minimum peak width.

Returns:

- `peaks` : Indices of detected peaks.
- `properties` : Additional information about detected peaks.

H. `plt.plot()`

- Creates a line plot.
- Syntax: `plt.plot(x, y)`

I. `plt.xlabel()` and `plt.ylabel()`

- `plt.xlabel("Frequency")` : Sets the X-axis label.
- `plt.ylabel("Magnitude")` : Sets the Y-axis label.

J. `plt.scatter()`

- Displays detected peaks as individual points.
- Useful for highlighting peak locations on a signal.

K. `plt.show()`

- Displays the signal and its detected peaks.

7. Sparse Matrices (`scipy.sparse`)

A. `csr_matrix()`

Purpose: Converts a matrix into Compressed Sparse Row (CSR) format.

Syntax:

```
csr = csr_matrix(A)
```

Parameters:

- `A` : Input dense matrix or compatible sparse data.

Importance:

- Stores non-zero values efficiently.
- Useful for row-based operations.
- Saves memory when matrices contain many zeros.

B. `csc_matrix()`

Purpose: Converts a matrix into Compressed Sparse Column (CSC) format.

Syntax:

```
csc = csc_matrix(A)
```

Parameters:

- `A`: Input matrix.

Importance:

- Stores non-zero values efficiently.
- Useful for column-based operations.
- Saves memory for large sparse matrices.

C. `toarray()`

Purpose: Converts a sparse matrix into a regular dense NumPy array.

Syntax:

```
csr.toarray()
```

Parameters:

- No required parameters.

Important:

- Converts sparse representation into dense representation.
- Can consume significant memory for large matrices.

D. `nnz`

Purpose: Returns the number of stored non-zero entries in a sparse matrix.

Syntax:

```
csr.nnz
```

Important:

- Counts stored non-zero elements.
- Helps understand matrix sparsity.